



Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering Department



Third Year

Analog Communications

Term Project

MATLAB implementation of a superheterodyne receiver

Name:	رؤى عاطف محمد عبدالمجيد
Sec:	2
ID:	9220291

Table of Contents

1. Introduction.....	5
2. System Components.....	5
2.1. Transmitter.....	5
2.1.1. Discussion:	5
2.1.2. Figures:	6
2.2. RF Stage (Receiver Stage).....	7
2.2.1. Discussion:	7
2.2.2. Figures :	8
2.3. Mixer Stage (Receiver Stage).....	8
2.3.1. Discussion:	8
2.3.2. Figures:	9
2.4. IF Stage (Receiver Stage).....	9
2.4.1. Discussion:	9
2.4.2. Figures:	10
2.5. Baseband Stage (Receiver Stage).....	10
2.5.1. Discussion:	10
2.5.2. Figures:	11
3. Role Of IF And RF And Baseband detector.....	12
4. Can we successfully listen to the radio station.....	12
5. Adding Noise	12
5.1. Effect of adding noise:	12
5.2. Figures:	13
6. Removing RF stage	15
6.1. Discussion:	15
6.2. Figures:	15
6.3. can we successfully listen to the radio station:	17
7. Offset of receiver oscillator	17
7.1. 0.2 kHz offset:.....	17
7.1.1. Spectrum:	17
7.1.2.Sound quality:	17

7.1.3. Figures:	17
7.2. 1.2 kHz offset:.....	19
7.2.1. Spectrum:	19
7.2.2. Sound quality:	19
7.2.3. Figures:	20
8. Matlab Code	22

Table of Figures

Figure 1:Communication system	5
Figure 2:AM block.....	5
Figure 3:Signal 2 before upsampling.....	6
Figure 4:Signal 1 before upsampling.....	6
Figure 5:Signal 1 after upsampling	6
Figure 6:Signal 2 after upsampling	6
Figure 7:Signal 2 after modulation	7
Figure 8:Signal 1 after modulation	7
Figure 9:FDM spectrum	7
Figure 10:RF block.....	7
Figure 11:Signal 2 after RF stage.....	8
Figure 12:Signal 1 after RF stage.....	8
Figure 13:Mixer block	8
Figure 14:Signal 2 after first mixer.....	9
Figure 15:Signal 1 after first mixer.....	9
Figure 16:IF block.....	9
Figure 17:Signal 2 after IF BPF	10
Figure 18:Signal 1 after IF BPF	10
Figure 19:Baseband block.....	10
Figure 20:Signal 2 after second mixer.....	11
Figure 21:Signal 1 after second mixer.....	11
Figure 22:Signal 2 after LPF.....	11
Figure 23:Signal 1 after LPF.....	11
Figure 24:Signal 2 after downsampling.....	11
Figure 25:Signal 1 after downsampling.....	11
Figure 26:Noise effect on signal 2 after RF stage	13
Figure 27:Noise effect on signal 1 after RF stage	13
Figure 28:Signal 2 after first mixer(with noise effect)	13

Figure 29:Signal 1 after first mixer(with noise effect)	13
Figure 30:Signal 2 after IF BPF (with noise effcet)	13
Figure 31:Signal 1 after IF BPF (with noise effect)	13
Figure 32:Signal 2 after second mixer(with noise effect)	14
Figure 33:Signal 1 after seocnd mixer (with noise effect)	14
Figure 34:Signal 2 after LPF (with noise effect)	14
Figure 35:Signal 1 after LPF (with noise effect)	14
Figure 36:Signal 2 after downsampling(with noise effect)	14
Figure 37:Signal 1 after downsampling(with noise effect)	14
Figure 38:Signal 2 after first mixer(after removing RF stage).....	15
Figure 39:Signal 1 after first mixer(after removing RF stage).....	15
Figure 40:Signal 2 after IF BPF (after removing RF stage).....	16
Figure 41:Signal 1 after IF BPF (after removing RF stage).....	16
Figure 42:Signal 2 after second mixer(after removing RF stage).....	16
Figure 43:Signal 1 after second mixer(after removing RF stage).....	16
Figure 44:Signal 2 after LPF (with removing RF stage)	16
Figure 45:Signal 1 after LPF (with removing RF stage)	16
Figure 46:Signal 2 after downsampling(after removing RF stage).....	17
Figure 47:Signal 1 after downsampling(after removing RF stage).....	17
Figure 48: Signal 2 after the first mixer (with offset 0.2 kHz)	18
Figure 49: Signal 1 after first mixer (with offset 0.2 kHz)	18
Figure 50:Signal 2 after IF BPF (with offset 0.2 kHz)	18
Figure 51:Signal 1 after IF BPF (with offset 0.2 kHz)	18
Figure 52: Signal 2 after the second mixer (with offset 0.2 kHz)	18
Figure 53: Signal 1 after the second mixer (with offset 0.2 kHz)	18
Figure 54:Signal 2 after LPF (with offset 0.2 kHz)	19
Figure 55:Signal 1 after LPF (with offset 0.2 kHz)	19
Figure 56:Signal 2 after downsampling (with offset 0.2 kHz)	19
Figure 57: Signal 1 after downsampling (with offset 0.2 kHz)	19
Figure 58:Signal 2 after first mixer (with offset 1.2 kHz)	20
Figure 59:Signal 1 after first mixer (with offset 1.2 kHz)	20
Figure 60:Signal 2 after IF BPF (with offset 1.2 kHz)	20
Figure 61:Signal 1 after IF BPF (with offset 1.2 kHz)	20
Figure 62: Signal 2 after the second mixer (with offset 1.2 kHz)	20
Figure 63: Signal 1 after the second mixer (with offset 1.2 kHz)	20
Figure 64:Signal 2 after LPF (with offset 1.2 kHz)	21
Figure 65:Signal 1 after LPF (with offset 1.2 kHz)	21
Figure 66:Signal 2 after downsampling (with offset 1.2 kHz)	21
Figure 67:Signal 1 after downsampling (with offset 1.2 kHz)	21

1. Introduction

This project simulates an analog communication system using MATLAB, focusing on an AM modulator and super-heterodyne receiver. The task involves processing audio signals through modulation, filtering, mixing, and baseband detection, as well as applying frequency-division multiplexing (FDM) to transmit multiple modulated audio channels. The project provides practical experience in signal processing and communication system design.

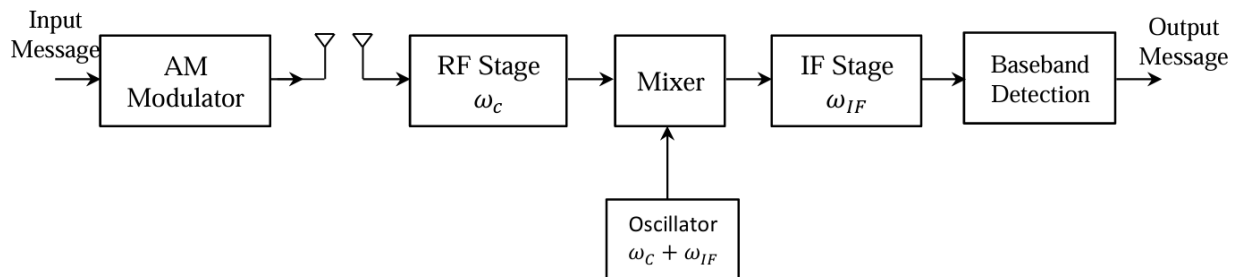


FIGURE 1: COMMUNICATION SYSTEM

2. System Components

2.1. Transmitter

2.1.1. Discussion:

A **Transmitter** uses **DSB-SC** modulation to encode and prepare data for transmission.

Steps In This Stage:

1. Convert to Mono: Input audio files are read with **Audioread**, and stereo signals are summed into Mono.

Reason: Reduces complexity by processing single-channel signals.

2. Match Sampling Rates and Lengths: Signals are resampled to the highest sampling rate (**fs = max(fs1, fs2)**) and zero-padded to equal lengths.

Reason: Ensures consistency and prevents data loss.

3. Upsampling: Signals are upsampled using an interpolation factor to increase their effective sampling rate.

Reason: Satisfies the Nyquist criterion and avoids aliasing during modulation.

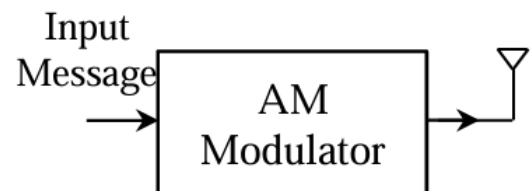


FIGURE 2: AM BLOCK

4.Modulate Signals: Signals are modulated with 100 kHz and 150 kHz carriers by multiplying with cosine waves.

Reason: Shifts signals to separate frequency bands for **FDM**.

5. Combine Signals (FDM): The modulated signals are added to create a single FDM signal for transmission.

2.1.2. Figures:

- Before UpSampling and Modulation

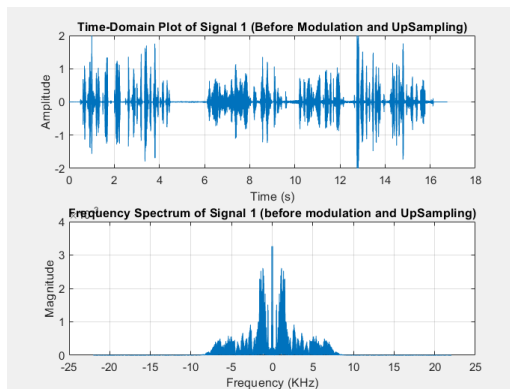


FIGURE 4: SIGNAL 1 BEFORE UPSAMPLING

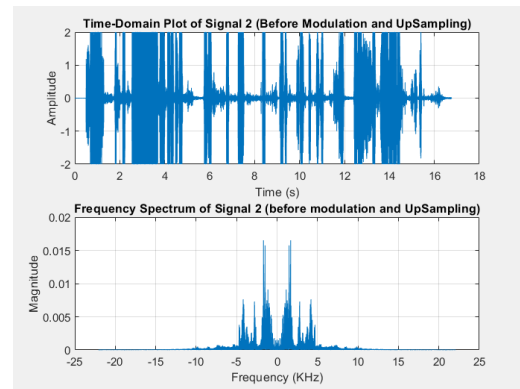


FIGURE 3: SIGNAL 2 BEFORE UPSAMPLING

- After UpSampling

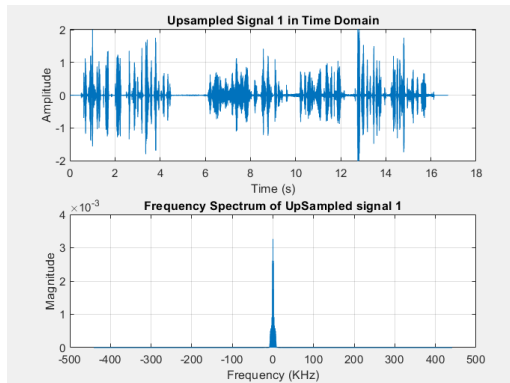


FIGURE 5: SIGNAL 1 AFTER UPSAMPLING

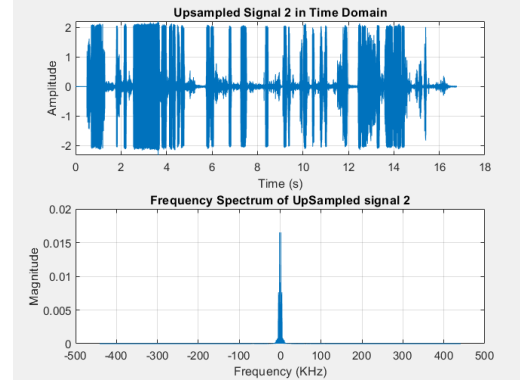


FIGURE 6: SIGNAL 2 AFTER UPSAMPLING

- After Modulation

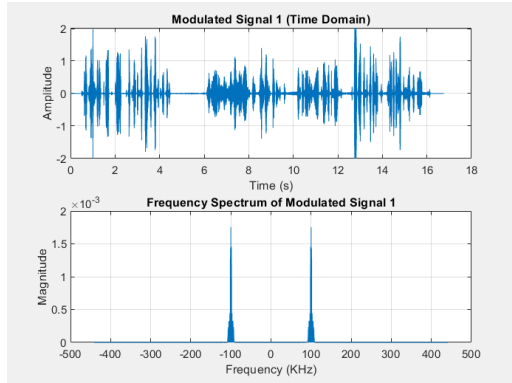


FIGURE 8: SIGNAL 1 AFTER MODULATION

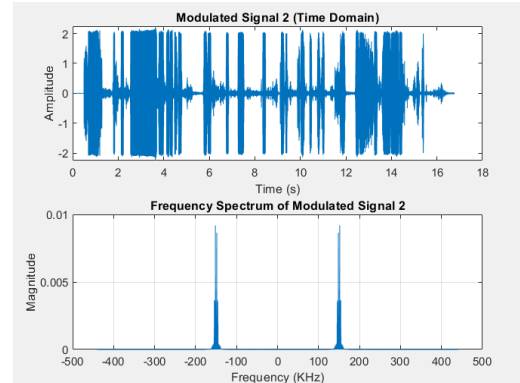


FIGURE 7: SIGNAL 2 AFTER MODULATION

○ FDM Signal

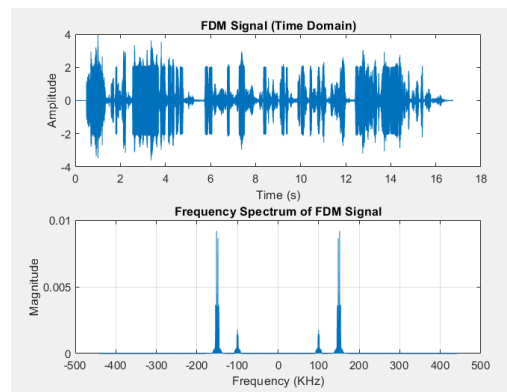


FIGURE 9: FDM SPECTRUM

2.2. RF Stage (Receiver Stage)

2.2.1. Discussion:

The **RF stage** in communication systems filters and isolates desired frequency components, often combined via Frequency Division Multiplexing (FDM), and prepares the signal for demodulation.

- I ask the user at the beginning of this stage to choose which signal to demodulate

Steps In This Stage:

1. Signal Reception: The received **FDM** signal contains modulated Signal 1 and Signal 2, each at different carrier frequencies.

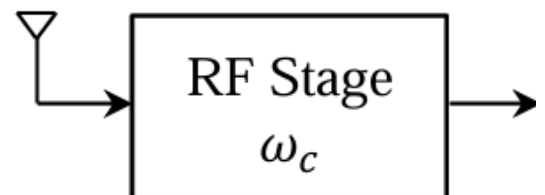


FIGURE 10: RF BLOCK

2. Band-Pass Filtering (BPF): A BPF isolates the frequency range of the selected signal (Signal 1 or Signal 2) by focusing on a narrow band around its carrier frequency, e.g., 100 kHz for Signal 1. The filter order is: **(8)**, cut-off1 is: **(carrier_freq+BW_{sig})**, cut-off2 is: **(carrier_freq-BW_{sig})**

3. Filtered Output: The BPF output retains only the frequency components around the carrier of the chosen signal.

2.2.2. Figures :

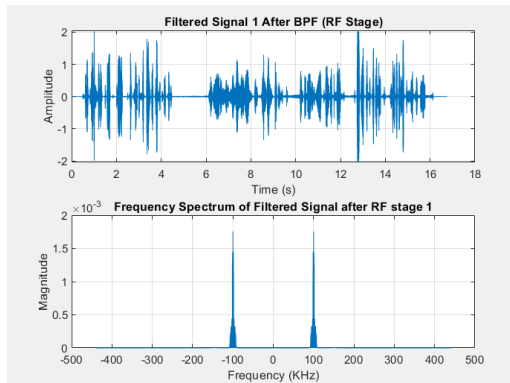


FIGURE 12: SIGNAL 1 AFTER RF STAGE

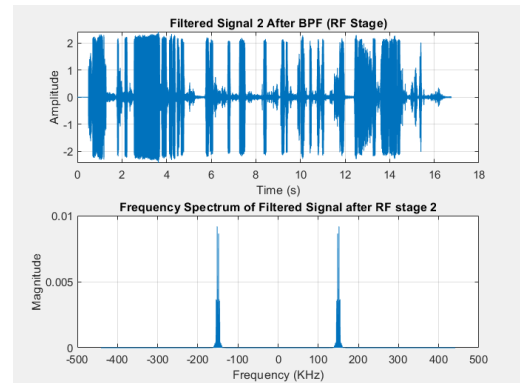


FIGURE 11: SIGNAL 2 AFTER RF STAGE

2.3. Mixer Stage (Receiver Stage)

2.3.1. Discussion:

The Mixer in radio communication systems shifts a signal to a different frequency band, typically an intermediate frequency (IF), by mixing the filtered signal with a local oscillator signal

Steps In This Stage:

1. Local Oscillator Frequency (ω_c): The local oscillator frequency is calculated as the carrier frequency of the chosen signal plus the intermediate frequency (IF). For Signal 1, $\omega_c = 100 \text{ kHz} + 25 \text{ kHz}$; for Signal 2, $\omega_c = 150 \text{ kHz} + 25 \text{ kHz}$.

2. Local Oscillator Generation: The oscillator signal is generated using $\cos(2\pi\omega_c t)$

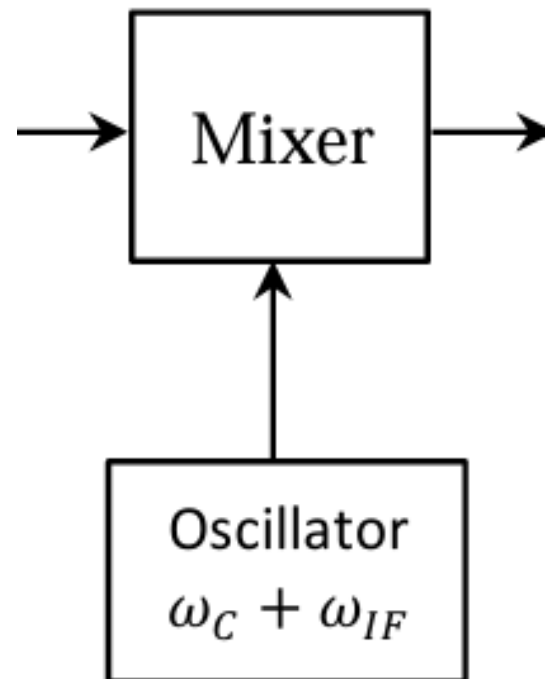


FIGURE 13: MIXER BLOCK

3. Signal Mixing: The filtered signal is multiplied with the local oscillator to shift its frequency to the intermediate frequency (**IF**), producing the **IF Signal=filteredSignal×localOscillator**.

2.3.2. Figures:

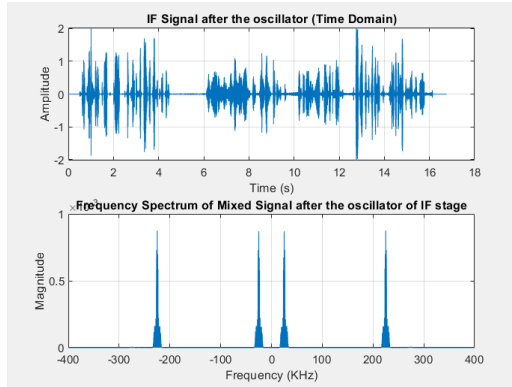


FIGURE 15: SIGNAL 1 AFTER FIRST MIXER

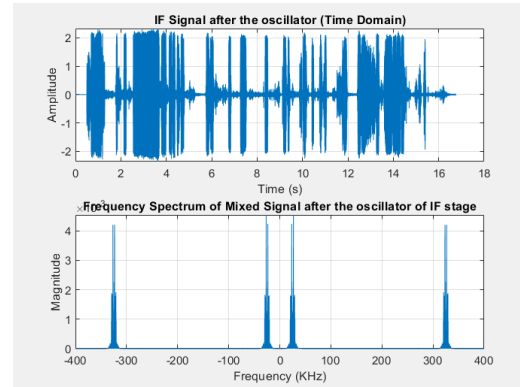


FIGURE 14: SIGNAL 2 AFTER FIRST MIXER

2.4. IF Stage (Receiver Stage)

2.4.1. Discussion:

The Intermediate Frequency (IF) Stage filters the mixed signal to isolate the **IF** component, enabling signal recovery at a manageable frequency range.

Steps In This Stage:

- Band-Pass Filtering (BPF) at IF:** A BPF centered around the intermediate frequency (**IF**) isolates the **IF** range, retaining signals within the desired bandwidth (e.g., 10 kHz) and attenuating unwanted components. filter order is **(6)**, **cut-off1** is: $(\omega_{IF} + BW_{sig})$, **cut-off2** is: $(\omega_{IF} - BW_{sig})$
- Filtered Output:** The resulting signal has frequency components centered around the **IF**, with unwanted frequencies removed.

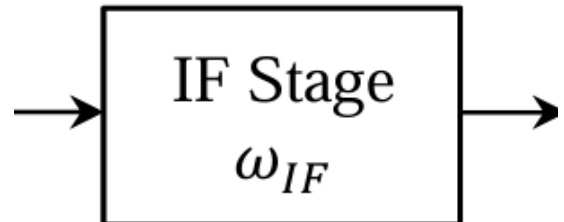


FIGURE 16: IF BLOCK

2.4.2.Figures:

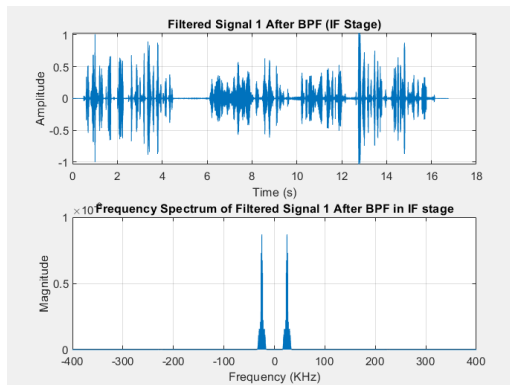


FIGURE 18: SIGNAL 1 AFTER IF BPF

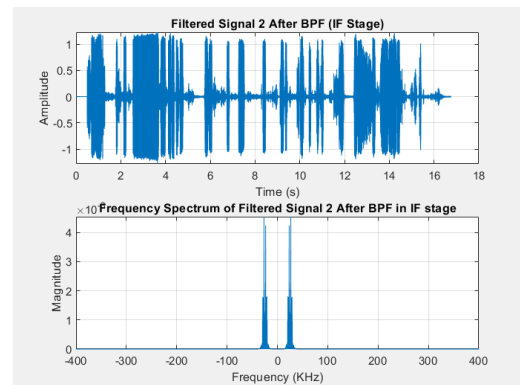


FIGURE 17: SIGNAL 2 AFTER IF BPF

2.5. Baseband Stage (Receiver Stage)

2.5.1. Discussion:

The **Baseband Detection Stage** is where the frequency-shifted signal is processed to bring it back to its baseband (zero frequency).

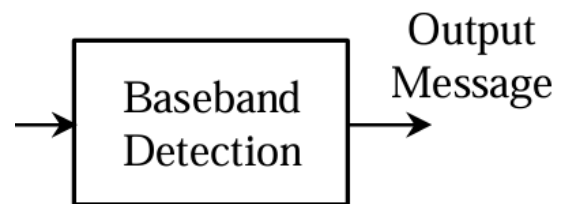


FIGURE 19: BASEBAND BLOCK

Steps In This Stage:

1. **Mixing with Local Oscillator:**

The signal is mixed with a local oscillator (**LO**) at the intermediate frequency ($\omega_{LO} = \omega_{IF}$) to shift it to baseband, centering frequencies around zero.

2. Low-Pass Filtering (LPF): An LPF isolates the baseband signal by removing high-frequency components, passing only frequencies below the cutoff. filter order is **(8)**, the **cut-off** is **(BW_{sig})**

3. Baseband Signal: The filtered baseband signal, typically in the audio range, is the final processed signal.

4. Downsampling Stage: The sample rate is reduced using LPF of order **12** and cut-off **fs/2** to avoid aliasing, followed by downsampling by a factor of **20**.

2.5.2. Figures:

- After Mixer

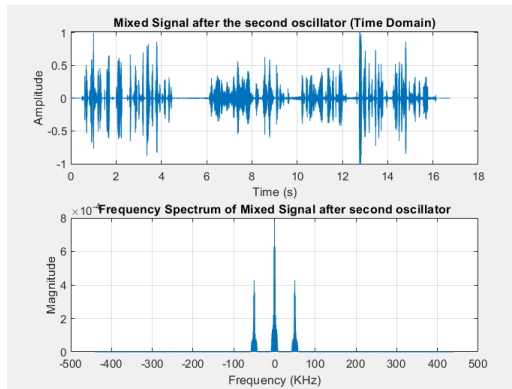


FIGURE 21: SIGNAL 1 AFTER SECOND MIXER

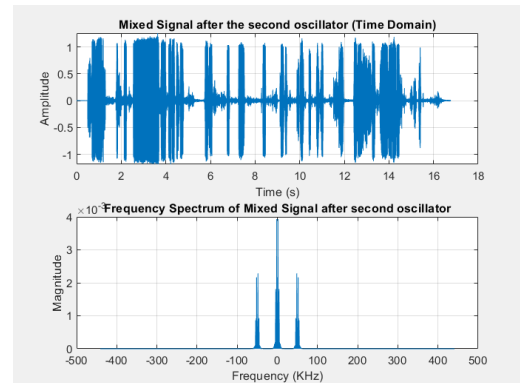


FIGURE 20: SIGNAL 2 AFTER SECOND MIXER

- After LPF

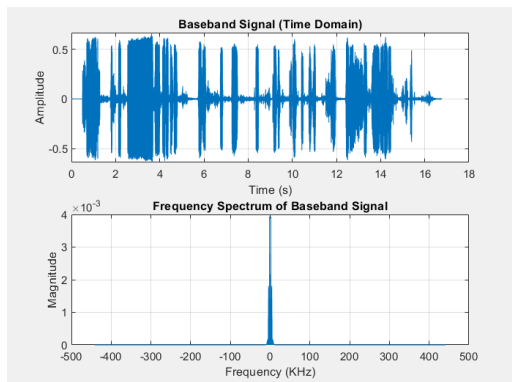


FIGURE 23: SIGNAL 1 AFTER LPF

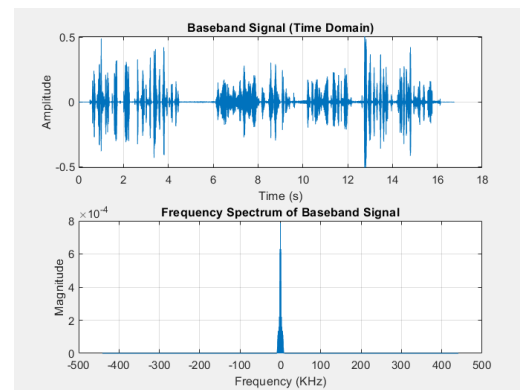


FIGURE 22: SIGNAL 2 AFTER LPF

- After DownSampling

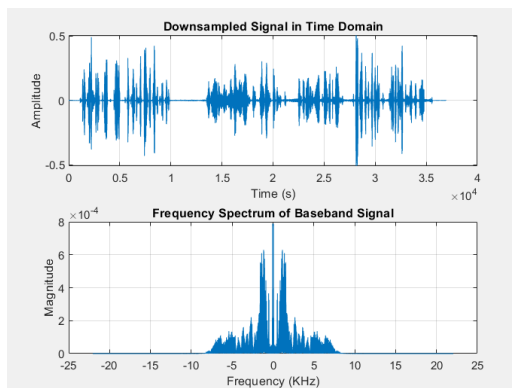


FIGURE 25: SIGNAL 1 AFTER DOWNSAMPLING

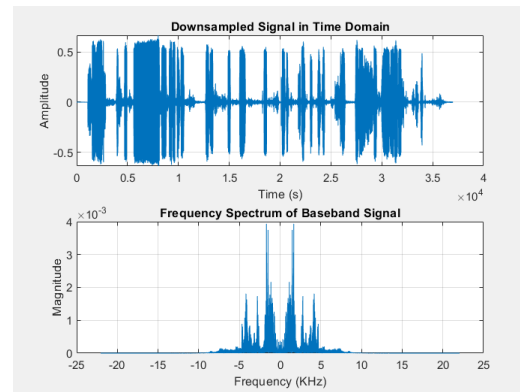


FIGURE 24: SIGNAL 2 AFTER DOWNSAMPLING

3. Role Of IF And RF And Baseband detector

The RF stage receives the transmitted signal, which is modulated at high frequencies for efficient long-distance transmission.

The IF stage downconverts the signal to a lower intermediate frequency to simplify filtering and amplification, as high-frequency signals are harder to process.

The baseband detector then converts the intermediate-frequency signal back to the original baseband signal through mixing with a local oscillator and low-pass filtering.

- **The IF stage** is crucial for improving signal processing by making it easier to filter noise and achieve better signal-to-noise ratios for accurate demodulation.

4. Can we successfully listen to the radio station

Yes, using (**sound**) function we can successfully listen to the baseband or the demodulated signal without any noise.

5. Adding Noise

5.1. Effect of adding noise:

When noise was added to the signal after the RF stage using the **AWGN** command, the output sound became unclear, and the noise was clearly noticeable in the spectrum. This distortion lowered the signal-to-noise ratio (**SNR**), significantly affecting the clarity of the sound. The added noise disrupted the original signal's integrity, making it harder to distinguish from the noise. This observation shows how noise introduced at this stage can affect the audio quality and overall performance of the system

Steps In This Stage:

1. A signal-to-noise ratio (**SNR**) of **2 dB** is added to the system after the **RF** stage to observe its effect on the signal.
2. Using the **AWGN** command, we merge the noise with the filtered signal output from the **RF** low-pass filter (**LPF**).

5.2. Figures:

- After Adding Noise After BPF Of RF Stage

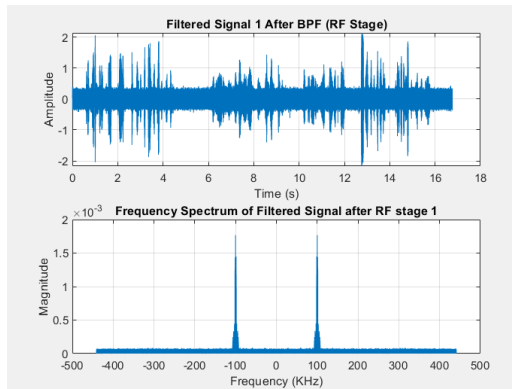


FIGURE 27: NOISE EFFECT ON SIGNAL 1 AFTER RF STAGE

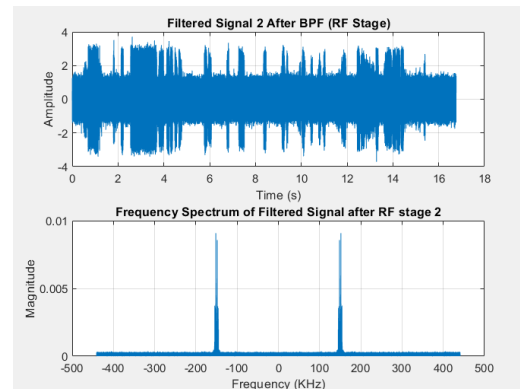


FIGURE 26: NOISE EFFECT ON SIGNAL 2 AFTER RF STAGE

- After IF oscillator

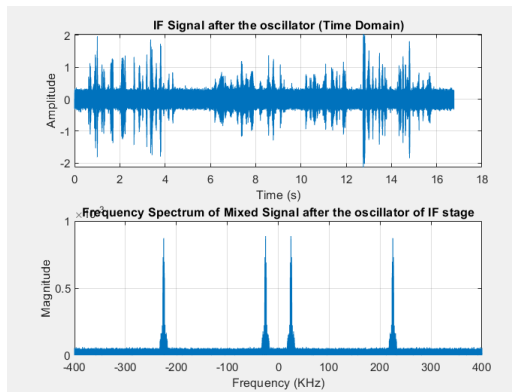


FIGURE 29: SIGNAL 1 AFTER FIRST MIXER (WITH NOISE EFFECT)

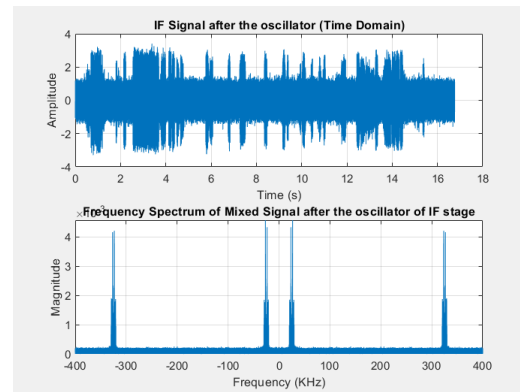


FIGURE 28: SIGNAL 2 AFTER FIRST MIXER (WITH NOISE EFFECT)

- After IF BPF

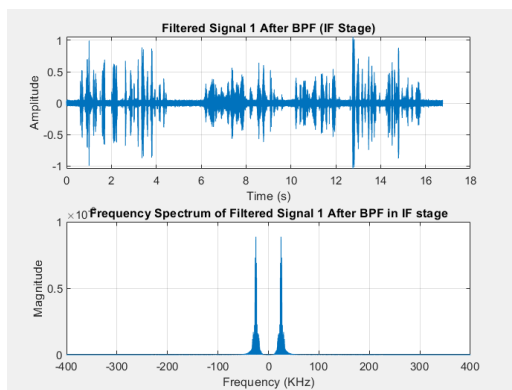


FIGURE 31: SIGNAL 1 AFTER IF BPF (WITH NOISE EFFECT)

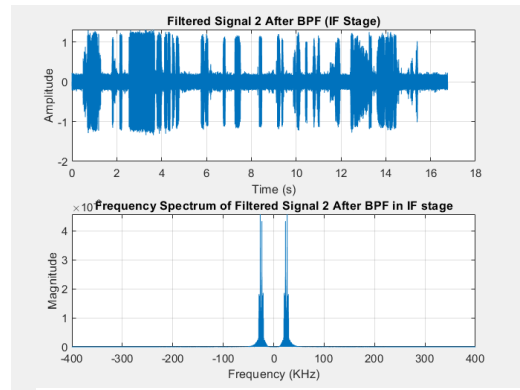


FIGURE 30: SIGNAL 2 AFTER IF BPF (WITH NOISE EFFECT)

- After baseband oscillator

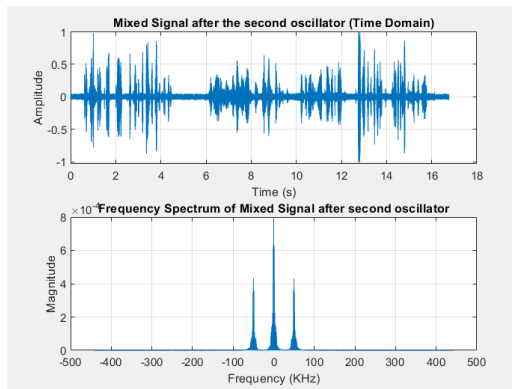


FIGURE 33: SIGNAL 1 AFTER SEOCND MIXER (WITH NOISE EFFECT)

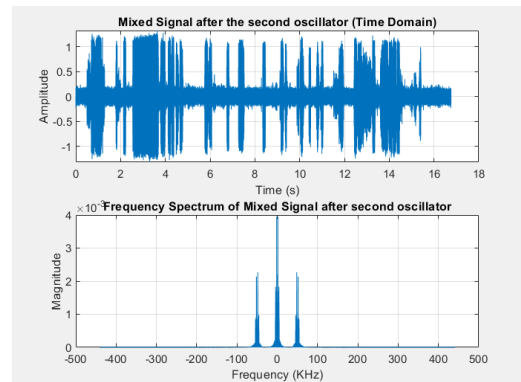


FIGURE 32: SIGNAL 2 AFTER SECOND MIXER (WITH NOISE EFFECT)

- After baseband LPF

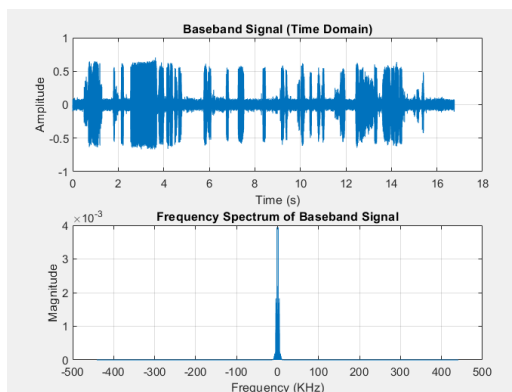


FIGURE 34: SIGNAL 1 AFTER LPF (WITH NOISE EFFECT)

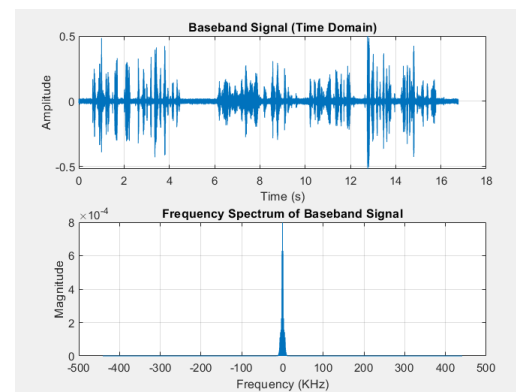


FIGURE 35: SIGNAL 2 AFTER LPF (WITH NOISE EFFECT)

- After DownSampling

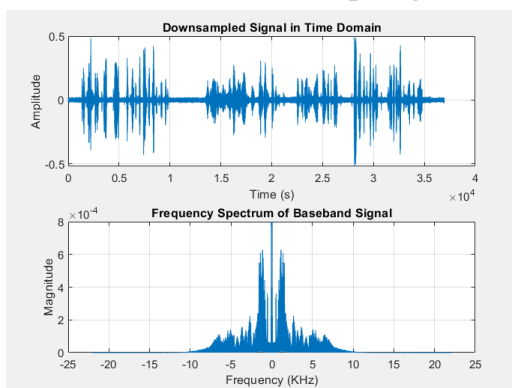


FIGURE 37: SIGNAL 1 AFTER DOWNSAMPLING (WITH NOISE EFFECT)

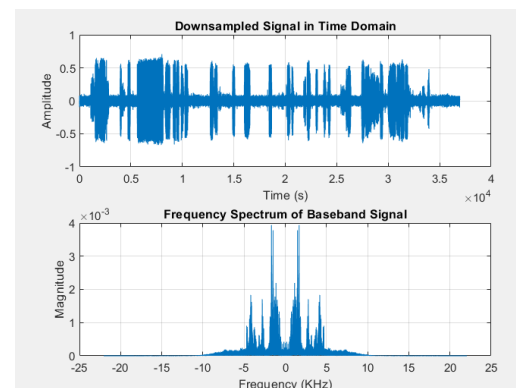


FIGURE 36: SIGNAL 2 AFTER DOWNSAMPLING (WITH NOISE EFFECT)

6. Removing RF stage

6.1. Discussion:

When both signals (Signal 1 and Signal 2) are present without proper filtering, their combined amplitudes and frequencies interfere with each other, causing aliasing, distortion, or interference in the demodulation stages. This makes it hard to cleanly separate the desired signal, degrading its quality or intelligibility.

Without an RF filter:

- Both signals, along with any noise, pass through without isolation.
- Without a filter to separate the signals, they overlap, causing interference.
- The mixed output will contain parts of both signals and potentially unwanted noise, resulting in intermodulation products and spectral distortion that corrupt the signal.

6.2. Figures:

- After the IF oscillator

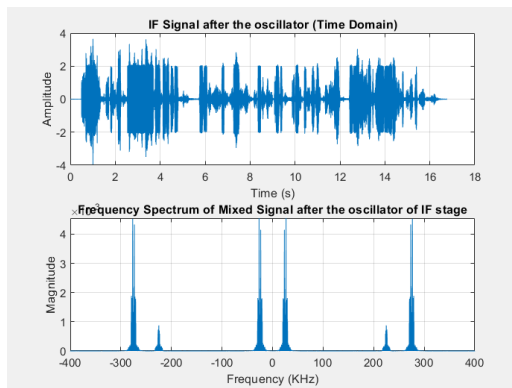


FIGURE 39: SIGNAL 1 AFTER FIRST MIXER (AFTER REMOVING RF STAGE)



FIGURE 38: SIGNAL 2 AFTER FIRST MIXER (AFTER REMOVING RF STAGE)

- After IF BPF

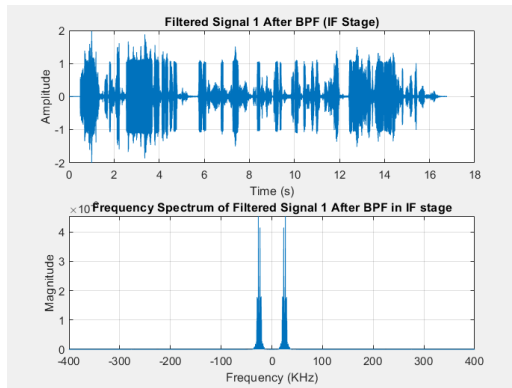


FIGURE 41: SIGNAL 1 AFTER IF BPF (AFTER REMOVING RF STAGE)

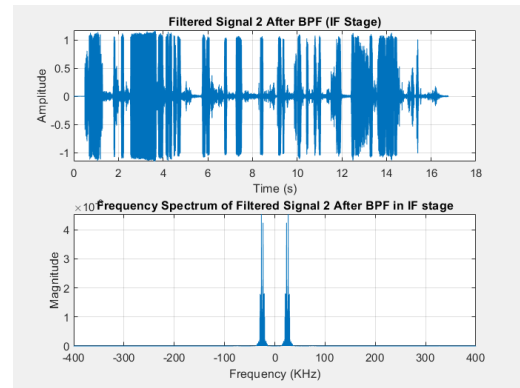


FIGURE 40: SIGNAL 2 AFTER IF BPF (AFTER REMOVING RF STAGE)

- After baseband oscillator

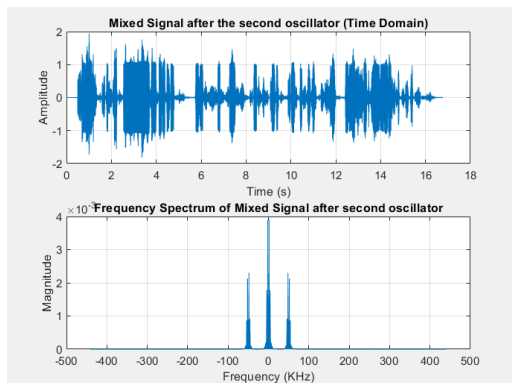


FIGURE 43: SIGNAL 1 AFTER SECOND MIXER (AFTER REMOVING RF STAGE)

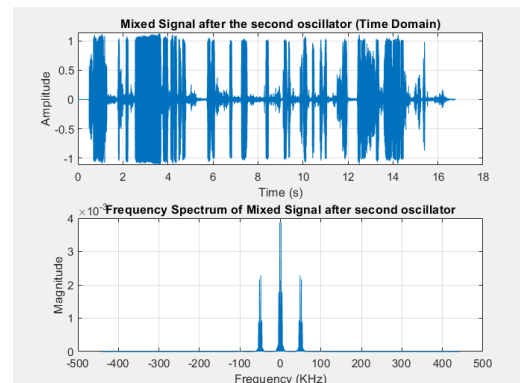


FIGURE 42: SIGNAL 2 AFTER SECOND MIXER (AFTER REMOVING RF STAGE)

- After baseband LPF

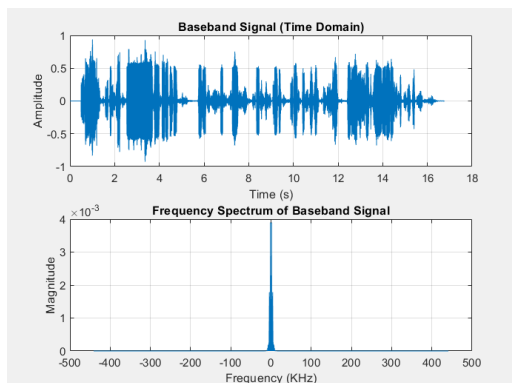


FIGURE 45: SIGNAL 1 AFTER LPF (WITH REMOVING RF STAGE)

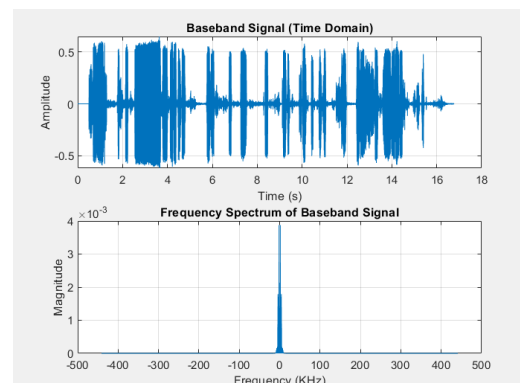


FIGURE 44: SIGNAL 2 AFTER LPF (WITH REMOVING RF STAGE)

- After DownSampling

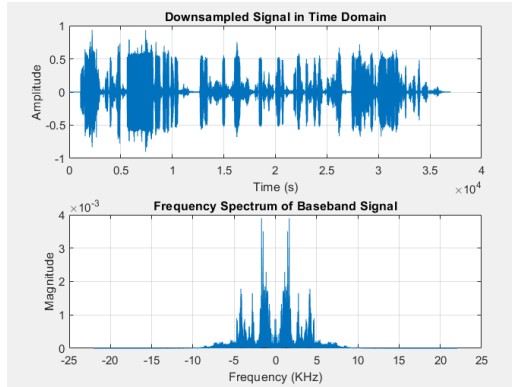


FIGURE 47: SIGNAL 1 AFTER DOWNSAMPLING(AFTER REMOVING RF STAGE)

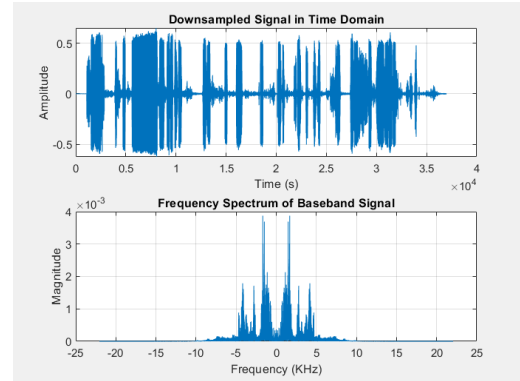


FIGURE 46: SIGNAL 2 AFTER DOWNSAMPLING(AFTER REMOVING RF STAGE)

6.3. can we successfully listen to the radio station:

When we use **sound** function to listen to the sound without the **RF** stage, The sound of the reconstructed signal would be heavily degraded. Artifacts such as noise, interference from the undesired signal, and spectral overlap would be audible.

7. Offset of receiver oscillator

7.1. 0.2 kHz offset:

7.1.1. Spectrum:

7.4.1. The entire frequency spectrum of the signal is shifted by **+0.2 kHz**. This means that every frequency component of the original signal is now slightly higher.

For instance, if the signal had a baseband frequency content ranging from **0** to **9 kHz**, it now shifts to **0.2 kHz** to **9.2 kHz**.

7.1.2. Sound quality:

This offset is small and does not change how the sound is heard very much but the sound may appear slightly "sharper" or "brighter" to the listener

7.1.3. Figures:

- After IF oscillator

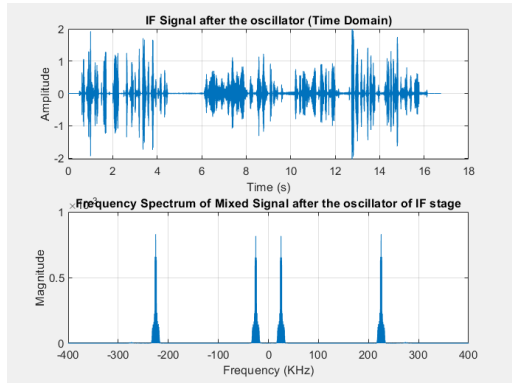


FIGURE 49: SIGNAL 1 AFTER FIRST MIXER (WITH OFFSET 0.2 kHz)

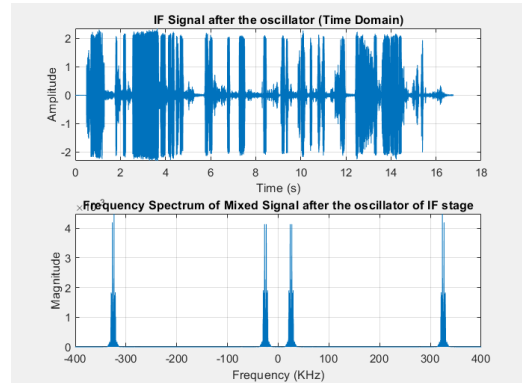


FIGURE 48: SIGNAL 2 AFTER THE FIRST MIXER (WITH OFFSET 0.2 kHz)

- After IF BPF

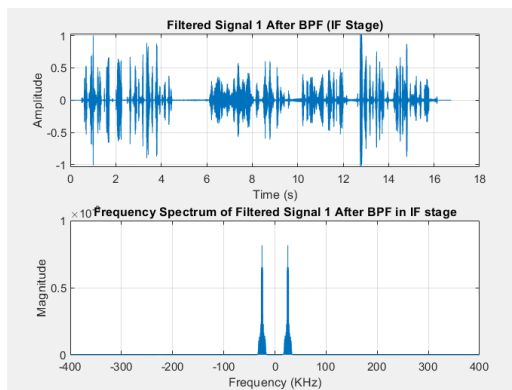


FIGURE 51: SIGNAL 1 AFTER IF BPF (WITH OFFSET 0.2 kHz)

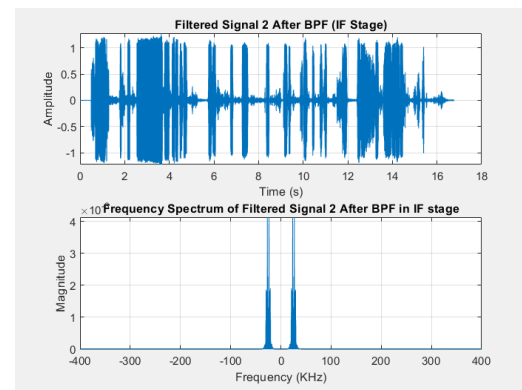


FIGURE 50: SIGNAL 2 AFTER IF BPF (WITH OFFSET 0.2 kHz)

- After baseband oscillator

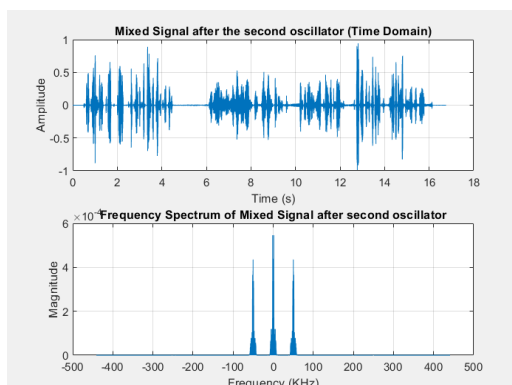


FIGURE 53: SIGNAL 1 AFTER THE SECOND MIXER (WITH OFFSET 0.2 kHz)

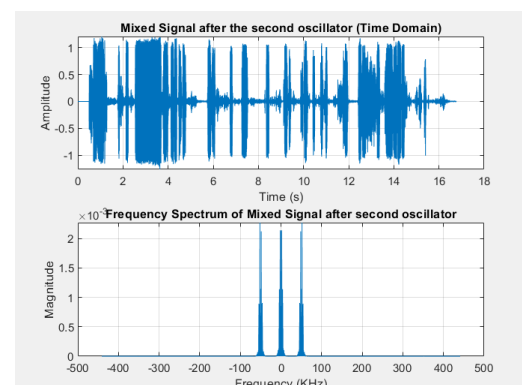


FIGURE 52: SIGNAL 2 AFTER THE SECOND MIXER (WITH OFFSET 0.2 kHz)

- After baseband LPF

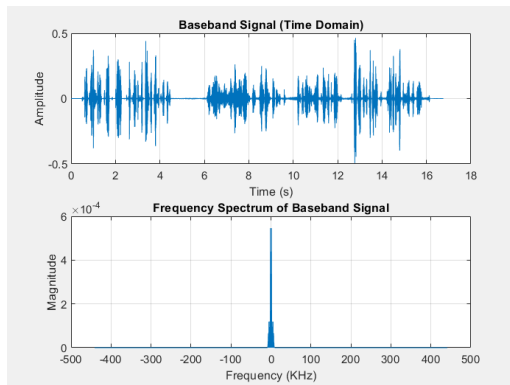


FIGURE 55: SIGNAL 1 AFTER LPF (WITH OFFSET 0.2 KHZ)

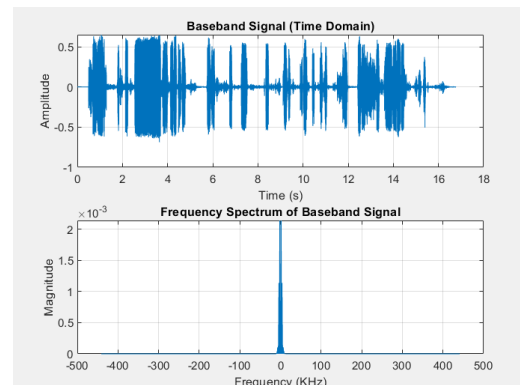


FIGURE 54: SIGNAL 2 AFTER LPF (WITH OFFSET 0.2 KHZ)

- After DownSampling

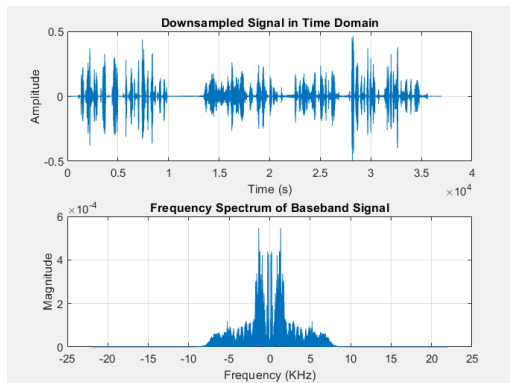


FIGURE 57: SIGNAL 1 AFTER DOWNSAMPLING (WITH OFFSET 0.2 KHZ)

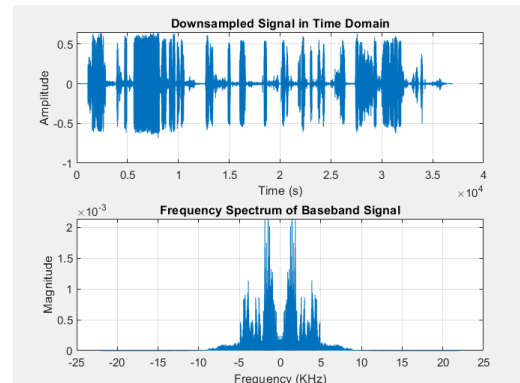


FIGURE 56: SIGNAL 2 AFTER DOWNSAMPLING (WITH OFFSET 0.2 KHZ)

7.2. 1.2 kHz offset:

7.2.1. Spectrum:

The spectrum is now shifted by a much larger amount compared to the 0.2 kHz offset. This shift moves the lower frequency components (**closer to 0 Hz**) into higher frequencies. As a result, the original "**low-frequency content**" of the signal is effectively **lost**, which can significantly affect the quality of the output.

7.2.2. Sound quality:

The higher shift causes noticeable changes to the sound, output sound sounds unnatural because the tones are shifted to a much higher range.

7.2.3. Figures:

- After IF oscillator

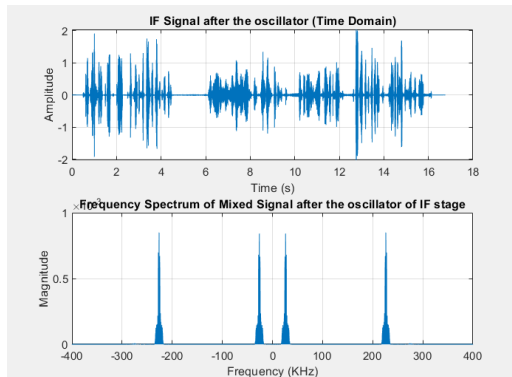


FIGURE 59: SIGNAL 1 AFTER FIRST MIXER (WITH OFFSET 1.2 kHz)

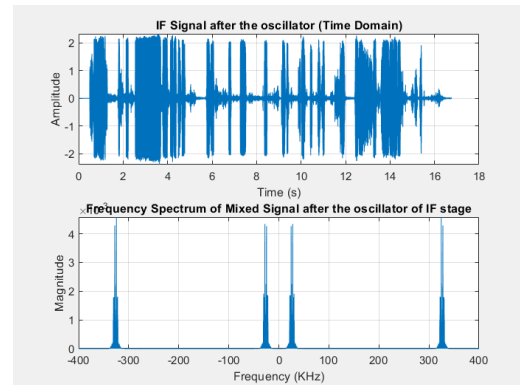


FIGURE 58: SIGNAL 2 AFTER FIRST MIXER (WITH OFFSET 1.2 kHz)

- After IF BPF

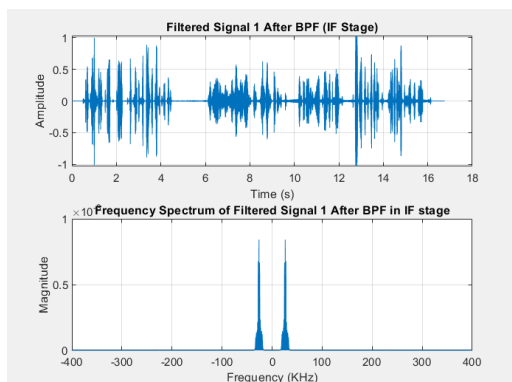


FIGURE 61: SIGNAL 1 AFTER IF BPF (WITH OFFSET 1.2 kHz)

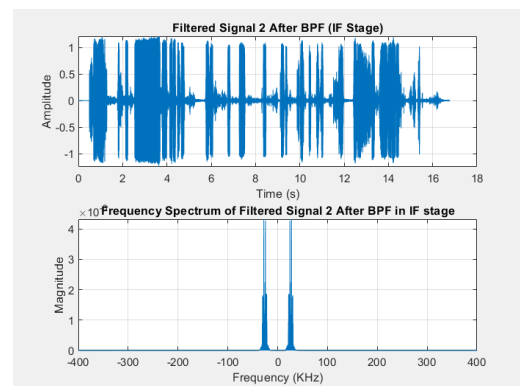


FIGURE 60: SIGNAL 2 AFTER IF BPF (WITH OFFSET 1.2 kHz)

- After baseband oscillator

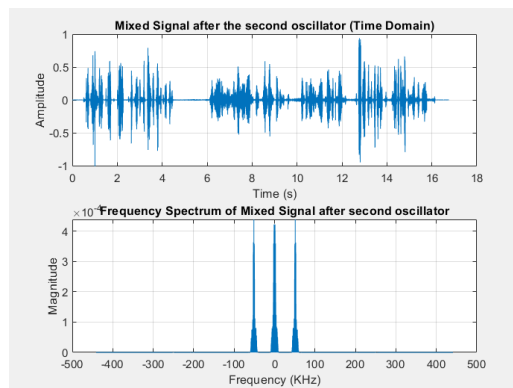


FIGURE 63: SIGNAL 1 AFTER THE SECOND MIXER (WITH OFFSET 1.2 kHz)

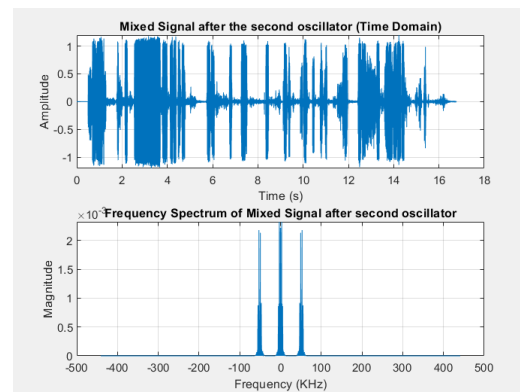


FIGURE 62: SIGNAL 2 AFTER THE SECOND MIXER (WITH OFFSET 1.2 kHz)

- After baseband LPF

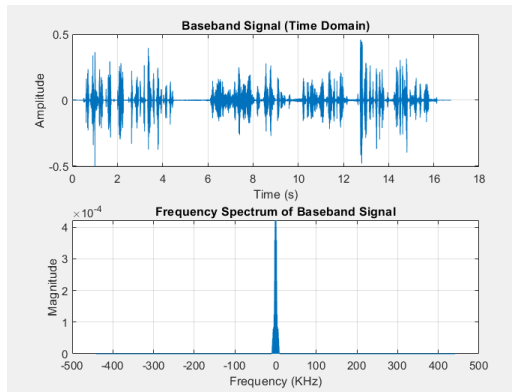


FIGURE 65: SIGNAL 1 AFTER LPF (WITH OFFSET 1.2 KHZ)

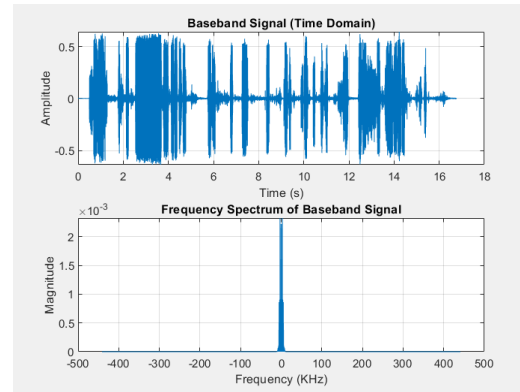


FIGURE 64: SIGNAL 2 AFTER LPF (WITH OFFSET 1.2 KHZ)

- After DownSampling

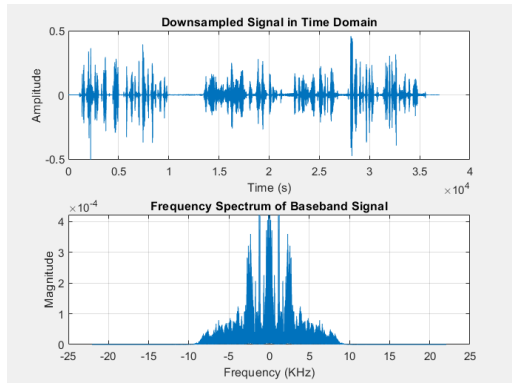


FIGURE 67: SIGNAL 1 AFTER DOWNSAMPLING (WITH OFFSET 1.2 KHZ)

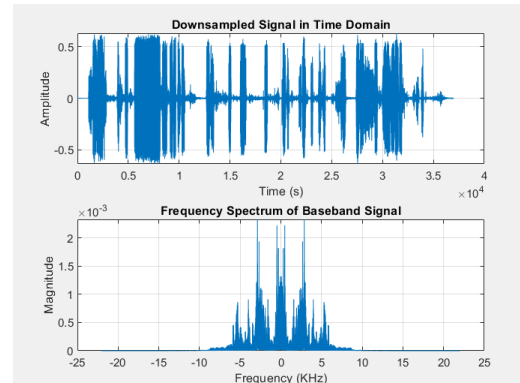


FIGURE 66: SIGNAL 2 AFTER DOWNSAMPLING (WITH OFFSET 1.2 KHZ)

8. Matlab Code

```
% Prepare Signals For The Operation
% Read the two signals

[signal1, fs1] = audioread('Short_SkyNewsArabia.wav'); % Read first audio file
[signal2 , fs2] = audioread('Short_QuranPalestine.wav'); % Read second audio file

% Get mono signals
mono_Signal1 = sum(signal1, 2); % Sum the channels for the first audio
mono_Signal2 = sum(signal2, 2); % Sum the channels for the second audio

% Match sampling rates and pad signals to equal length
fs = max(fs1, fs2); % Use the higher sampling rate
signal1 = resample(mono_Signal1, fs, fs1); %resampling for signal 1
signal2 = resample(mono_Signal2, fs, fs2); % resampling for signal 2

% Equalize the lengths of the two signals
maxLength = max(length(signal1), length(signal2));
signal1 = [signal1; zeros(maxLength - length(signal1), 1)];
signal2 = [signal2; zeros(maxLength - length(signal2), 1)];
% Plotting before modulation
% Plot in time and frequency domain for signal 1
t = (0:maxLength-1)/fs; % Time axis for Signal 1 and signal 2
N1 = length(signal1);
FFT_Signal1 = fft(signal1); % Compute FFT
fftShifted1 = fftshift(FFT_Signal1); % Shift zero frequency to the center
frequencies1 = linspace(-fs/2, fs/2, N1)/1000; % Frequency axis
figure;
subplot(2,1,1);
plot(t, signal1);
title('Time-Domain Plot of Signal 1 (Before Modulation and UpSampling)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(2,1,2);
plot(frequencies1, (abs(fftShifted1)/N1));
title('Frequency Spectrum of Signal 1 (before modulation and UpSampling)');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;
% Plot in time domain and frequency domain For Signal 2
figure;
subplot(2,1,1);
plot(t, signal2);
title('Time-Domain Plot of Signal 2 (Before Modulation and UpSampling)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
```

```

N2 = length(signal2);
FFT_Signal2 = fft(signal2);
fftShifted2 = fftshift(FFT_Signal2);
frequencies2 = linspace(-fs/2, fs/2, N2)/1000;

subplot(2,1,2);
plot(frequencies2, (abs(fftShifted2)/N2));
title('Frequency Spectrum of Signal 2 (before modulation and UpSampling)');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;
%% Transmitter Stage
% Modulating the signals
F_C = [100e3, 150e3]; % Carrier frequencies for two signals

% Upsampling signals to meet Nyquist criteria
up_factor = 20; % Increase sampling frequency by 20x
signal1_up = interp(signal1, up_factor);
signal2_up = interp(signal2, up_factor);
Fs_up = fs * up_factor;
t_up = (0:length(signal1_up)-1)/Fs_up;

% Plotting In Time Domain and frequency domain for signal 1
N1 = length(signal1_up);
fft_Signal1 = fft(signal1_up); % Compute FFT
fft_Shifted1 = fftshift(fft_Signal1); % Shift zero frequency to the center
frequencies1 = linspace(-Fs_up/2, Fs_up/2, N1)/1000; % Frequency axis
figure;
subplot(2,1,1);
plot(t_up, signal1_up);
title('Upsampled Signal 1 in Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

subplot(2,1,2);
plot(frequencies1, (abs(fft_Shifted1)/N1));
title('Frequency Spectrum of UpSampled signal 1');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;
% Plotting In Time Domain and frequency domain for signal 2
N2 = length(signal2_up);
fft_Signal2 = fft(signal2_up); % Compute FFT
fft_Shifted2 = fftshift(fft_Signal2); % Shift zero frequency to the center
frequencies2 = linspace(-Fs_up/2, Fs_up/2, N2)/1000; % Frequency axis
figure;
subplot(2,1,1);
plot(t_up, signal2_up);
title('Upsampled Signal 2 in Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

```

```

subplot(2,1,2);
plot(frequencies1, (abs(fft_Shifted2)/N2));
title('Frequency Spectrum of UpSampled signal 2');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;
% Generate carriers
carrier1 = cos(2 * pi * F_C(1) * t_up); % Carrier for Signal 1
carrier2 = cos(2 * pi * F_C(2) * t_up); % Carrier for Signal 2

% Modulate signals
modulatedSignal1 = signal1_up .* carrier1;
modulatedSignal2 = signal2_up .* carrier2;

% FDM Signal
FDM_Signal = modulatedSignal1 + modulatedSignal2;

% Plotting the modulated signal 1
N1 = length(modulatedSignal1);
FFT_Signal1 = fft(modulatedSignal1); % Compute FFT
fftShifted1 = fftshift(FFT_Signal1); % Shift zero frequency to the center

frequencies1 = linspace(-Fs_up/2, Fs_up/2, N1)/1000; % Frequency axis

figure;
subplot(2,1,1);
plot(t_up, modulatedSignal1);
title('Modulated Signal 1 (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
subplot(2,1,2);
plot(frequencies1, (abs(fftShifted1)/N1));
title('Frequency Spectrum of Modulated Signal 1');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;
% Plotting Modulated Signal 2
N2 = length(modulatedSignal2);
FFT_Signal2 = fft(modulatedSignal2); % Compute FFT
fftShifted2 = fftshift(FFT_Signal2); % Shift zero frequency to the center
frequencies2 = linspace(-Fs_up/2, Fs_up/2, N2)/1000; % Frequency axis

figure;
subplot(2,1,1);
plot(t_up, modulatedSignal2);
title('Modulated Signal 2 (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(2,1,2);
plot(frequencies2, (abs(fftShifted2)/N2));
title('Frequency Spectrum of Modulated Signal 2');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;

```



```

%For FDM
N_FDM = length(FDM_Signal);
FFT_FDM = fft(FDM_Signal); % Compute FFT of FDM signal
fftShifted_FDM = fftshift(FFT_FDM); % Shift zero frequency to the center
frequencies_FDM = linspace(-Fs_up/2, Fs_up/2, N_FDM)/1000; % Frequency axis

figure;
subplot(2,1,1);
plot(t_up, FDM_Signal);
title('FDM Signal (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(2,1,2);
plot(frequencies_FDM, (abs(fftShifted_FDM)/N_FDM));
title('Frequency Spectrum of FDM Signal');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;
%% THE RECEIVER STAGE
%% 1.RF STAGE
% Assume FDM_Signal is the received signal at the receiver side

% Carrier Frequencies from the Transmitter

% User chooses which signal to demodulate (Signal 1 or Signal 2)
signal_choice = input('Enter 1 to demodulate Signal 1, or 2 to demodulate Signal 2: ');

% Select the carrier frequency based on user's choice
if signal_choice == 1
    carrier_freq_signal = F_C(1); % Carrier frequency for Signal 1
    signal_to_filter = FDM_Signal; % Signal 1 is part of FDM signal
elseif signal_choice == 2
    carrier_freq_signal = F_C(2); % Carrier frequency for Signal 2
    signal_to_filter = FDM_Signal; % Signal 2 is part of FDM signal
else
    error('Invalid input! Please enter 1 for Signal 1 or 2 for Signal 2.');
```

end

```

% if we removed RF stage we will remove from 1 to 2 and pass FDM signal to the mixer directly
and we will remove part 3
% 1
% Design the Band-Pass Filter (BPF) for the selected signal
filter_order = 8; % Order of the filter (adjustable)
bandwidth = 9e3; % Bandwidth around the carrier (adjustable)

% Design a Band-Pass Filter centered at 'carrier_freq_signal' with the specified bandwidth
BPF_signal = designfilt('bandpassiir', 'FilterOrder', filter_order, ...
    'HalfPowerFrequency1', carrier_freq_signal - bandwidth, ...
    'HalfPowerFrequency2', carrier_freq_signal + bandwidth, ...
    'SampleRate', Fs_up); % fs is the sample rate used in the signal

% Apply the Band-Pass Filter to isolate the chosen signal
filteredSignal = filter(BPF_signal, signal_to_filter); % Apply BPF to the selected signal
% 2

```

```

%% Adding noise

% Add White Gaussian Noise (AWGN) after the RF Stage
%SNR = 2; % Specify the Signal-to-Noise Ratio in dB (adjustable)
%filteredSignal = awgn(filteredSignal, SNR, 'measured'); % Add noise to the filtered signal

%% Plot The Filtered Signal After RF BPF In Time Domain and Frequency Domain
% part 3
figure;
subplot(2,1,1);
plot(t_up, filteredSignal);
title(['Filtered Signal ' num2str(signal_choice) ' After BPF (RF Stage)']);
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

N = length(filteredSignal); % Number of samples in filtered signal
FFT_filteredSignal = fft(filteredSignal); % Compute FFT of filtered signal
fftShifted = fftshift(FFT_filteredSignal); % Shift zero frequency to the center
frequencies = linspace(-Fs_up/2, Fs_up/2, N)/1000; % Frequency axis

subplot(2,1,2);
plot(frequencies, (abs(fftShifted)/N)); % Plot magnitude of FFT (frequency spectrum)
title(['Frequency Spectrum of Filtered Signal after RF stage ' num2str(signal_choice)]);
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;

%% 2.MIXER STAGE

omega_IF = 25e3; % Intermediate frequency (IF)
carrier_freq_signal = F_C(signal_choice); % Carrier frequency selected based on user's choice

% Calculate the local oscillator frequency omega_c
omega_c = carrier_freq_signal + omega_IF; % Local oscillator frequency (carrier + IF)

% Generate the local oscillator signal (cosine wave)
localOscillator = cos(2 * pi * omega_c * t_up); % Local oscillator signal at omega_c
%% Adding offset
% add 0.2 khz offset
%localOscillator = cos(2 * pi * (omega_c+200) * t_up); % Local oscillator signal at
omega_c+offset
% add 1.2 khz offset
% localOscillator = cos(2 * pi * (omega_c+1200) * t_up); % Local oscillator signal at
omega_c+offset
%% Mix the filtered signal with the local oscillator (simple multiplier)
IF_Signal = filteredSignal .* localOscillator; % Perform the mixing

% Plotting The Mixed Signal In The Time Domain and frequency domain
figure;
subplot(2,1,1);
plot(t_up, IF_Signal);
title('IF Signal after the oscillator (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

```

```

N_IF = length(IF_Signal); % Number of samples in mixed signal
FFT_IFSignal = fft(IF_Signal); % Compute FFT of the mixed signal
fftShifted_IF = fftshift(FFT_IFSignal); % Shift zero frequency to the center
frequencies_IF = linspace(-Fs_up/2, Fs_up/2, N_IF)/1000; % Frequency axis

subplot(2,1,2);
plot(frequencies_IF, (abs(fftShifted_IF)/N_IF)); % Plot magnitude of FFT (frequency spectrum)
title('Frequency Spectrum of Mixed Signal after the oscillator of IF stage');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
xlim([-400 400]); % Set x-axis range to -400 kHz to 400 kHz
grid on;
%% 3. IF Stage: Band-Pass Filter (BPF) centered at intermediate frequency (omega_IF)
filter_order = 6; % Order of the filter (adjustable)

% Design a Band-Pass Filter centered at omega_IF
BPF_IF = designfilt('bandpassiir', 'FilterOrder', filter_order, ...
    'HalfPowerFrequency1', omega_IF - 9e3, ... % Adjust bandwidth
    'HalfPowerFrequency2', omega_IF + 9e3, ...
    'SampleRate', Fs_up); % fs is the sample rate used in the signal

% Apply the Band-Pass Filter to isolate the IF signal
filtered_IF_Signal = filter(BPF_IF, IF_Signal); % Apply BPF to the selected signal

% Plot The Time-Domain and frequency domain Signal

figure;
subplot(2,1,1);
plot(t_up, filtered_IF_Signal);
title(['Filtered Signal ' num2str(signal_choice) ' After BPF (IF Stage)']);
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

N = length(filtered_IF_Signal); % Number of samples in filtered signal
FFT_filtered_IF_Signal = fft(filtered_IF_Signal); % Compute FFT of filtered signal
fftShifted_IF = fftshift(FFT_filtered_IF_Signal); % Shift zero frequency to the center
frequencies_IF = linspace(-Fs_up/2, Fs_up/2, N)/1000; % Frequency axis

subplot(2,1,2);
plot(frequencies_IF, (abs(fftShifted_IF)/N)); % Plot magnitude of FFT (frequency spectrum)
title(['Frequency Spectrum of Filtered Signal ' num2str(signal_choice) ' After BPF in IF
stage']);
xlabel('Frequency (KHz)');
ylabel('Magnitude');
xlim([-400 400]);
grid on;
%% 4. Baseband Detection Stage: Mixing with Local Oscillator and Low-Pass Filtering
% Mix with a carrier of suitable frequency to bring the signal down to baseband

% Local oscillator frequency (same as carrier frequency for signal 1 or 2)
omega_LO = omega_IF; % Local oscillator frequency

% Generate the local oscillator signal (cosine wave)
localOscillator = cos(2 * pi * omega_LO * t_up); % Local oscillator signal at omega_LO

```

```

% Mix the filtered signal with the local oscillator (simple multiplier)
mixedSignal = filtered_IF_Signal .* localOscillator; % Perform the mixing

% Plotting The Mixed Signal In The Time Domain and frequency domain

figure;
subplot(2,1,1);
plot(t_up, mixedSignal);
title('Mixed Signal after the second oscillator (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

N_mixed = length(mixedSignal); % Number of samples in mixed signal
FFT_mixedSignal = fft(mixedSignal); % Compute FFT of the mixed signal
fftShifted_mixed = fftshift(FFT_mixedSignal); % Shift zero frequency to the center
frequencies_mixed = linspace(-Fs_up/2, Fs_up/2, N_mixed)/1000; % Frequency axis

subplot(2,1,2);
plot(frequencies_mixed, (abs(fftShifted_mixed)/N_mixed)); % Plot magnitude of FFT (frequency spectrum)
title('Frequency Spectrum of Mixed Signal after second oscillator');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;
% Design the Low-Pass Filter (LPF) for baseband detection
baseband_cutoff = 9e3; % Cutoff frequency just above IF to filter out high frequencies
lpf_order = 8; % Order of the low-pass filter (adjustable)

% Design the Low-Pass Filter
LPF_baseband = designfilt('lowpassiir', 'FilterOrder', lpf_order, ...
    'HalfPowerFrequency', baseband_cutoff, ...
    'SampleRate', Fs_up);

% Apply the Low-Pass Filter to the mixed signal to obtain the baseband signal
basebandSignal = filter(LPF_baseband, mixedSignal);

% Plot The Time-Domain Baseband Signal and frequency domain
figure;
subplot(2,1,1);
plot(t_up, basebandSignal);
title('Baseband Signal (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;
N_baseband = length(basebandSignal); % Number of samples in baseband signal
FFT_basebandSignal = fft(basebandSignal); % Compute FFT of the baseband signal
fftShifted_baseband = fftshift(FFT_basebandSignal); % Shift zero frequency to the center
frequencies_baseband = linspace(-Fs_up/2, Fs_up/2, N_baseband)/1000; % Frequency axis

subplot(2,1,2);
plot(frequencies_baseband, (abs(fftShifted_baseband)/N_baseband)); % Plot magnitude of FFT (frequency spectrum)
title('Frequency Spectrum of Baseband Signal');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;

```

```

%% DownSampling
Down_F = up_factor; % Ratio of original to new sampling frequency

% Anti-Aliasing LPF Design
AntiAlias_Filter = designfilt('lowpassfir', ...
    'FilterOrder', 12, ...
    'HalfPowerFrequency', (fs / 2), ... % Nyquist frequency of original
sampling rate
    'SampleRate', Fs_up);

% Downsample Signal
DownSampledSignal1 = filter(AntiAlias_Filter, basebandSignal); % Apply anti-aliasing LPF
downsampled_signal = downsample(DownSampledSignal1, Down_F);
t_down = (0:length(downsampled_signal)-1) / Down_F; % Time vector for the downsampled signal

% Plot the Downsampled Signal in the Time Domain and frequency domain
figure;
subplot(2,1,1);
plot(t_down, downsampled_signal);
title('Downsampled Signal in Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');
grid on;

frequencies_ds = linspace(-fs/2, fs/2, length(downsampled_signal))/1000;

subplot(2,1,2);
plot(frequencies_ds, (abs(fftshift(fft(downsampled_signal)))/length(downsampled_signal)));
title('Frequency Spectrum of Baseband Signal');
xlabel('Frequency (KHz)');
ylabel('Magnitude');
grid on;
%% 5. Play Reconstructed Signal
sound(downsampled_signal / max(abs(downsampled_signal)), fs);

```